Verifying the SATA ΔC_0 correction (1-D)

Three checks on a single point target, with math and code

SATA / multichannel azimuth reconstruction

September 2026

The problem and the constants

A multichannel HRWS radar has N_{rx} receivers with a cross-track baseline. When a target is higher than the flat earth the reconstruction assumes, each channel measures a slightly different range — the residual ΔC_0. **SATA** puts that ΔC_0 back into each channel's phase before reconstruction. The checks ask whether the correction lands on the **right cell** (1), with the **right value** (2), and over the **whole target** so it lowers the ambiguities (3). Preset:

$$\lambda = 0.25\,\text{m},\ r_0 = 766.206\,\text{km},\ v = 7500\,\text{m/s},\ N_{rx} = 4,\ \text{PRF} = 2000\,\text{Hz},\ \text{PRF}_{\text{op}} = \frac{\text{PRF}}{N_{rx}} = 500\,\text{Hz}.$$

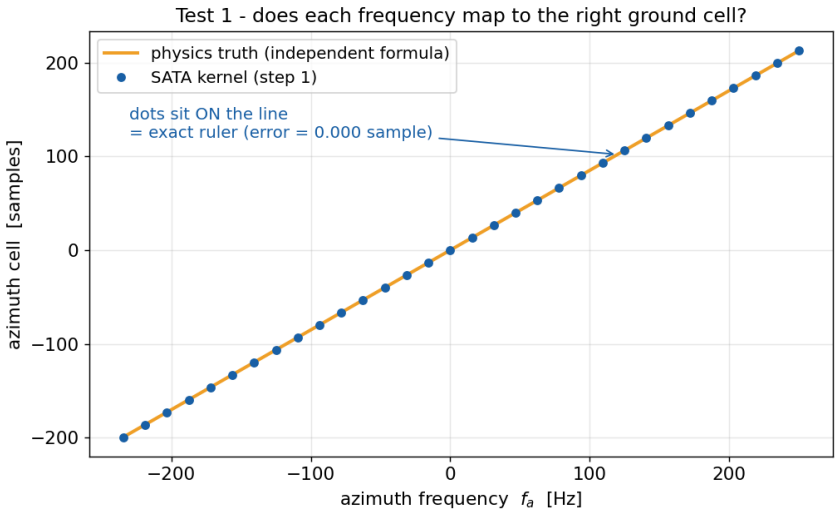
The correction engine (used by Test 3)

For each frequency bin the kernel turns a cell into a phase and applies it:

```
posaux = np.clip(np.round(azpos + centre).astype(int), 0, dimx-1) # bin -> cell
ph = -2*np.pi/wl * delta_C0_array[posaux] # read dC0 at the cell -> phase
spec *= np.exp(-1j*ph) if inverse else np.exp(1j*ph) # apply to the bin
```

i.e. $\varphi_j = -\frac{2\pi}{\lambda}\Delta C_0[\text{posaux}_j]$. The tests check that `posaux` and `delta_C0_array` are right.

Test 1 — does each frequency map to the right ground cell?



The axes, from scratch.

- **X:** azimuth (Doppler) frequency f_a [Hz] — each frequency is a look angle, and each angle is a ground position.
- **Y: the azimuth *cell*, in samples.** The azimuth image is a row of numbered boxes (one per sample); the target sits in box 256. Y counts those boxes (relative to broadside $= 0$), not metres or Hz. 1 sample $= v/\mathrm{PRF}_{\mathrm{op}} = 15$ m.

Numeric example. At $f_a = 250$ Hz: $\beta = \arcsin\left(\frac{0.25 \cdot 250}{2 \cdot 7500}\right) = 0.24°$; ground position $r_0 \tan\beta \approx 3195$ m; in boxes $3195/15 \approx$ **213**.

The two curves. Both use the same geometry $\beta \rightarrow$position; they differ in the *inputs*.

- **Orange (truth):** a direct, independent formula using the real sampling rate ($\mathrm{PRF}_{\mathrm{op}}$) and squint 0.
- **Blue (the kernel):** STEP 1 of SATA run for real — it builds its own frequency axis (`fsub`) and computes `azpos` from the arguments passed. Each dot is $(\texttt{fsub}[k], \texttt{azpos}[k])$.

Math. Both compute the SAME quantity: the Y value (the azimuth cell, in samples).

$$\underbrace{x_{\mathrm{truth}}(f) = r_0 \frac{\tan(\arcsin\frac{\lambda f}{2v})}{v}\mathrm{PRF}_{\mathrm{op}}}_{\text{orange: clean } f,\ \text{real PRF, sq=0}} \qquad \underbrace{\mathrm{azpos}_k = r\frac{\tan\beta_k}{v}\mathrm{prf},\ \beta_k = \arcsin\frac{\lambda\,\mathrm{fsub}_k}{2v}}_{\text{blue: kernel axis \& args}}$$

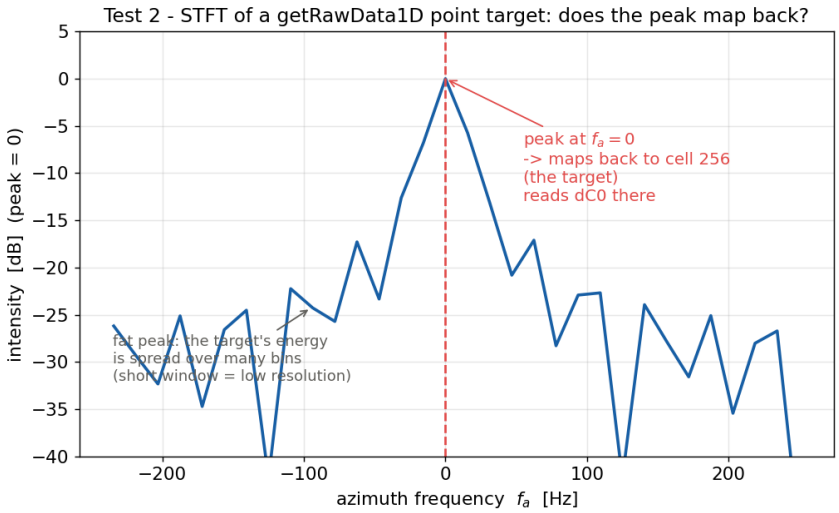
Code.

```
def truth_azpos(f): # ORANGE (independent)
   return r0*np.tan(np.arcsin(wl*f/(2*v)))/v*PRF_op
fsub = (np.arange(Nzp)*prf/Nzp - prf/2) + dfc # BLUE: kernel builds the axis
betasub = np.arcsin(wl*fsub/(2*v))
azpos = r*(np.tan(betasub) - np.tan(squint))/v*prf
error = azpos - truth_azpos(fsub) # what Test 1 compares
```

How this plot PROVES the test passes

Called correctly (`prf=PRF_op`, `Nsb=1`, `squint=0`) the blue dots sit on the orange line, `error=` 0.000 sample. Since the middle formula is shared, the test is not checking the trig — it confirms the argument passing, the axis construction (`dfc`, `np.roll`) and the squint, so that SATA later reads `delta_C0` at the right cell. **Dots on the line** $=$ **Test 1 passes.**

Test 2 — take the STFT peak back to the target (ΔC_0 reverse-engineering)



The axes, from scratch.

- **X:** azimuth frequency f_a [Hz].
- **Y: intensity in dB** — how strong each frequency is in the windowed signal, $Y(f) = 20\log_{10}(|\text{spec}(f)|/\max|\text{spec}|$ 0 dB is the strongest bin, -20 dB is $10\times$ weaker.

What the curve shows. The STFT of ONE point target (signal from the repo's `getRawData1D`, exact range history): a broad bump (the "fat peak"), centred at $f_a = 0$ (broadside = zero Doppler). It is fat because the window is short ($N_{zp} = 32$ bins, $\Delta f = 15.6$ Hz): the target's energy spreads over many bins.

Measured vs planted. Planted: the target is at cell 256 and the map holds ΔC_0 there. Measured: take the STFT peak, walk back through the ruler to a cell, read ΔC_0 there. **Math.**

$$Y(f) = 20\log_{10}\frac{|\text{spec}(f)|}{\max|\text{spec}|}, \quad k^* = \arg\max_j |\text{spec}_j|, \quad \underbrace{\text{cell} = \text{azpos}_{k^*} + \text{centre}}_{=256}.$$

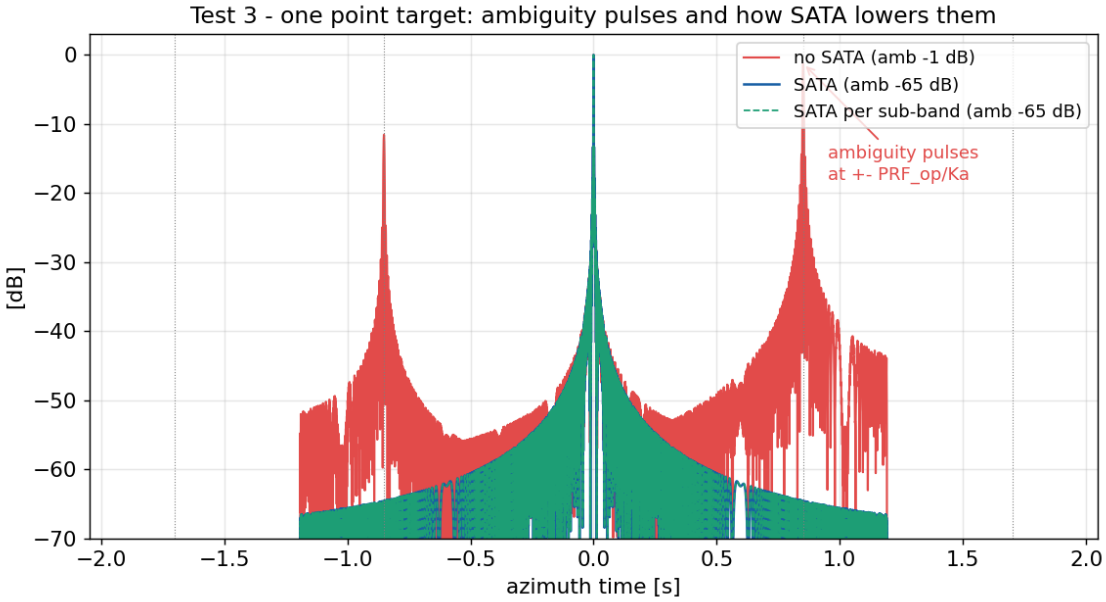
Code.

```
s = getRawData1D(ptg, ptx, ptx, vtx, vtx, ta, 0,0, theta,theta, wl, PRFop)
buf = s[start:start+2*Tsub]; buf = np.pad(buf, (0, Nzp-len(buf)))
spec = np.fft.fft(buf) # the STFT (the "fat peak")
kpk = np.argmax(np.abs(spec)) # peak bin
cell = round(azpos[kpk] + centre) # walk back through the ruler -> 256
```

How this plot PROVES the test passes

The peak is at $f_a = 0 \Rightarrow$ `cell`$= 256$ (the target cell). Starting from a real signal, the pipeline recovers the target position; Test 1 checked the ruler in the abstract, Test 2 checks it **end to end**. **Peak $\rightarrow$ target cell = Test 2 passes.**

Test 3 — the ambiguities and why one point is not enough



The axes, from scratch.

- **X:** azimuth time [s] — the focused image; the target is at $t = 0$. The full simulated scene (± 2.05 s) is shown; the signal only occupies the target's illuminated aperture ($\sim \pm 1.2$ s, the antenna beam), and is zero outside it.
- **Y: intensity in dB** (peak $= 0$). Each curve is one way to correct.

What the curve shows. The reconstructed IRF of ONE point target. Besides the peak at $t = 0$ there are **ambiguities** at $t = \pm \mathrm{PRF}_{\mathrm{op}}/K_a \approx \pm 0.85$ s: the replicas $f_a \pm n\,\mathrm{PRF}$ (each a different β), because each channel samples below Nyquist.

No approximations. The signal comes from the exact range history (`getRawData1D`); ΔC_0 is computed like the pipeline (below), not from a closed form. One point target (multi-target and α are the next step).

How ΔC_0 is computed (the pipeline way). Fit the range history to a polynomial about closest approach, take the constant term C_0 (stationary-phase result); the residual is the true target minus the flat point at the same range:

$$\Delta C_0^{(i)} = C_0(\text{true target, height } h) - C_0(\text{flat point, same } r), \qquad C_0 = \Delta c_0 + \frac{c_1^2}{4c_2} - \frac{(c_1+\Delta c_1)^2}{4(c_2+\Delta c_2)},$$

with $r_{ms} = 2|p_{tx} - p| \approx c_0 + c_1 t + c_2 t^2$ and $r_{bs} - r_{ms} \approx \Delta c_0 + \Delta c_1 t + \Delta c_2 t^2$. It is the repo's `residual_C0_3d`. Here $\Delta C_0^{(i)} = [0, -8.59, -17.19, -25.79]$ cm.

Three ways to correct.

- **no SATA:** the residual stays $\Rightarrow$ high ambiguities (-1 dB).
- **SATA:** correct the whole band $\Rightarrow$ ambiguities to the floor (-65 dB).
- **SATA per sub-band:** correct each sub-band with its $\beta_k \Rightarrow -65$ dB, *identical* to whole-band (for one target ΔC_0 is angle-invariant, so the two overlap — the documented case-matrix result).

Math. Ambiguity location and level; the ruler ties each replica to a β:

$$t_{\mathrm{amb}} = \pm n \frac{\mathrm{PRF}_{\mathrm{op}}}{K_a}, \; K_a = \frac{2v^2}{\lambda r_0}, \quad \mathrm{IRF} = \left|\mathrm{IFFT}(\mathrm{FFT}(s_{\mathrm{rec}})\,\overline{\mathrm{FFT}(\mathrm{ideal})})\right|, \quad f_a \pm n\,\mathrm{PRF} \to \beta.$$

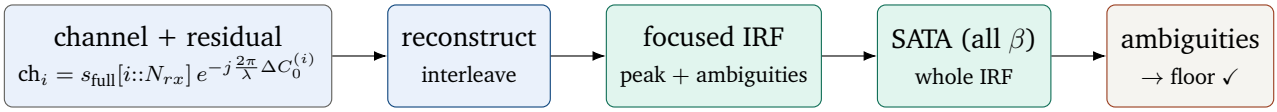
Code.

```
s_full = getRawData1D(ptg, ptx, ptx, vtx, vtx, ta, 0,0, theta,theta, wl, PRFfull)
dC0[i] = residual_C0_3d(p, tracks, ptg_real, channel=i) # C0(true)-C0(flat), real fit
ch[i] = s_full[i::Nrx]*np.exp(-1j*2*np.pi/wl*dC0[i]) # channel with residual
# SATA whole-band OR per sub-band (split into Nrx beta_k slices), then reconstruct
irf = np.fft.ifft(np.fft.fft(interleave(ch))*np.conj(np.fft.fft(s_full)))
```

How this plot PROVES the test passes

no SATA leaves the ambiguities at -1 dB; **SATA** (and **SATA per sub-band**, on top of it) take them to the floor -65 dB. This is "you cannot correct only one point ... the reason of the sub-bands": cover the whole IRF (all β). For a single target whole = per-sub-band; the split earns its cost only when the topography varies along azimuth. **Test 3 passes.**

Test 3 path:



Summary. **Test 1** — the f_a $\rightarrow$cell ruler matches physics. **Test 2** — the STFT peak of a `getRawData1D` target walks back to the target cell. **Test 3** — the target's ambiguities drop to the floor when SATA corrects the whole IRF (all β); whole-band = per-sub-band for one target. Common engine: $\varphi = -\frac{2\pi}{\lambda}\Delta C_0[\text{posaux}]$.

Signal from the repo's `getRawData1D`; coefficients from `get_coeff_nu_3d/residual_C0_3d`; SATA kernel is Prats' `sata_for_topography` with a corrected overlap-add. Figures from `plot_test{1,2,3}_*.py`.